HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 08

PR Automation, Quality Gates & LLM-as-Judge

Make pull-request review faster and more consistent without lowering the bar: AI review with evidence, quality gates in CI, and an LLM judge you can calibrate and trust, with a human always closing the loop.



DURATION

1 hour



FORMAT

Concepts · 1 lab



PRIMARY TOOL

Copilot code review + CI

How We'll Spend This Hour

From why review needs help, to gates, to judging the reviewer itself.



01

AI-Assisted PR Review

8 min

Where AI review fits in the pipeline and what it should look at.



02

Evidence & Configuration

10 min

Evidence-based comments, confidence, and configuring Copilot review with instructions and templates.



03

Quality Gates in CI

10 min

Layered gates, blocking versus advisory checks, and real-world gate recipes.



04

LLM-as-Judge

12 min

Rubrics, known biases, calibration against human labels, and escalation.



05

Lab: Review a Flawed Change

15 min

Run AI review and a judge on a deliberately flawed PR and capture the evidence.



06

Best Practices & Wrap-up

5 min

Do and avoid, then the handoff to no-code agents in Module 09.

Module 08 in the 12-Module Journey

Agents now write and review code; this module makes sure someone checks the checkers.

FOUNDATION			BUILD & VALIDATE			EXTEND WITH TOOLS & AGENTS				GOVERN	CAPSTONE
1	2	3	4	5	6	7	**8**	9	10	11	12
Objectives & GenAI Recap	Copilot Advanced	Prompt & Context Eng.	Spec-Driven Dev (Spec Kit)	SDD to Full SDLC	Test Strategy & Validation	MCP & Custom MCP Server	**PR Automation & LLM-Judge**	No-Code Agents (Rovo)	Code Agents & Simple RAG	Observability, ROI & Gov.	Capstone & Roadmap

M04–05
The spec is what review checks the change against

M06
Test evidence and regression risk come from the test strategy

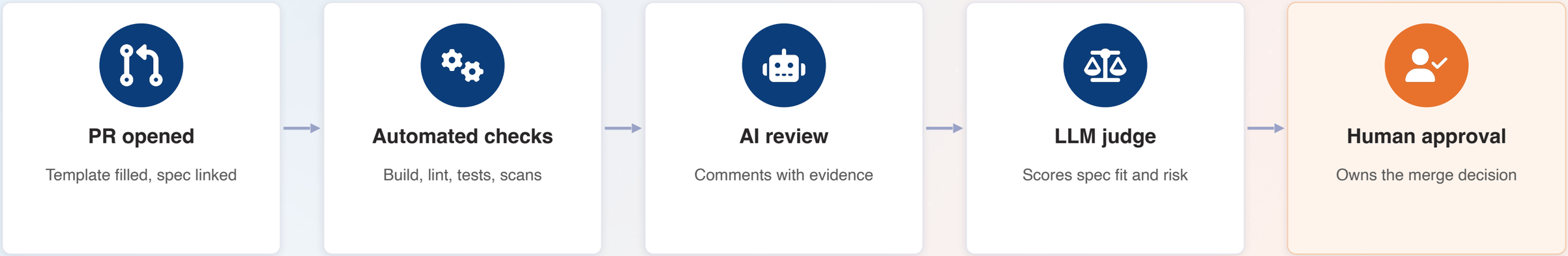
M07
MCP tools feed review agents; the same governance applies

M11
Judge scores and gate results become quality metrics

Why this matters: AI raises the volume of code faster than humans can read it. Review has to get smarter, not just faster.

The Review Pipeline: Automation First, Human Last

Each stage removes a kind of problem so the human reviewer spends time on judgement.



Deterministic ←————————————→ *Probabilistic* ←——→ *Accountable*

Principle: AI review and judges advise; they do not own the decision. Merge approval stays with a named human, and required CI checks, not AI comments, are what block a merge.

What an AI Reviewer Should Check

Six dimensions — the first two are the ones generic review tools miss.

Specification compliance

Does the change meet each acceptance criterion in spec.md?

Coding and design standards

Naming, layering, patterns your team agreed — written down.

Security checks

Input handling, secrets, authorisation, risky dependencies.

Test evidence

Do tests exist for the new behaviour, and do they actually assert it?

Regression risk

What else calls this code? What existing behaviour could change?

Maintainability

Complexity, duplication, readability for the next person.

Real-world use case: a change passes CI and looks tidy, but silently drops an acceptance criterion. Only a spec-aware review catches that; a linter never will.

Evidence-Based Comments, Confidence and Escalation

A comment you cannot verify costs more reviewer time than it saves.

WEAK COMMENT

“This might have security issues. Consider improving error handling.”

No location, no evidence, no action, no confidence.

EVIDENCE-BASED COMMENT (illustrative)

“OrderService.cs L42: AC-3 requires rejecting quantity ≤ 0, but this path accepts 0. No test covers it. Suggest a guard and a boundary test. Confidence: high.”

Where	What	Why	Fix	Confidence
File and line	Observed behaviour	Spec / standard / risk	Suggested action	High · medium · low

Escalation rule: low confidence, security findings, or anything touching a protected area goes to a named human reviewer. Agree the rule once as a team; do not leave it to each reviewer.

Configuring AI Review with Instruction Files

Tell Copilot code review what your team cares about — in short, specific, testable rules.

REPO-WIDE AND PATH-SPECIFIC RULES

.github/instructions/api.instructions.md (illustrative)

```
---
applyTo: "src/api/**"
---
# API review rules
- Every endpoint must validate input and
  return the error shape in docs/errors.md.
- Flag any change that lacks a test for
  each new acceptance criterion (AC-n).
- Never log tokens, emails or full payloads.
- Prefer small functions; flag methods > 40
  lines and say why it hurts readability.
```

copilot-instructions.md

Repository-wide rules.

***.instructions.md + applyTo**

Rules for specific paths, in .github/instructions.

Read from the PR head branch

Test an instruction change in the same PR.

Keep it small

Start with 10–20 specific rules; iterate on real PRs.

Limits to teach: instructions cannot change comment formatting, block merges, or follow external links, and vague goals like “be more accurate” do nothing. Behaviour is non-deterministic, so test and refine.

PR Templates and Quality Checklists: Give the Reviewer Evidence

A good template makes the author supply what the reviewer — human or AI — needs.

.github/pull_request_template.md (illustrative)

```
## What and why
Spec: specs/orders/spec.md (AC-1..AC-4)

## Evidence
- [ ] Tests added/updated (list AC IDs)
- [ ] CI green; test report linked
- [ ] No new secrets or PII in logs

## Risk
- Callers affected: <list>
- Rollback: <how>

## AI assistance
- Tools used and what was human-reviewed
```

Spec linked

Reviewer knows what “done” means

Tests mapped to AC

Evidence, not a claim

Risk and rollback

Blast radius is stated

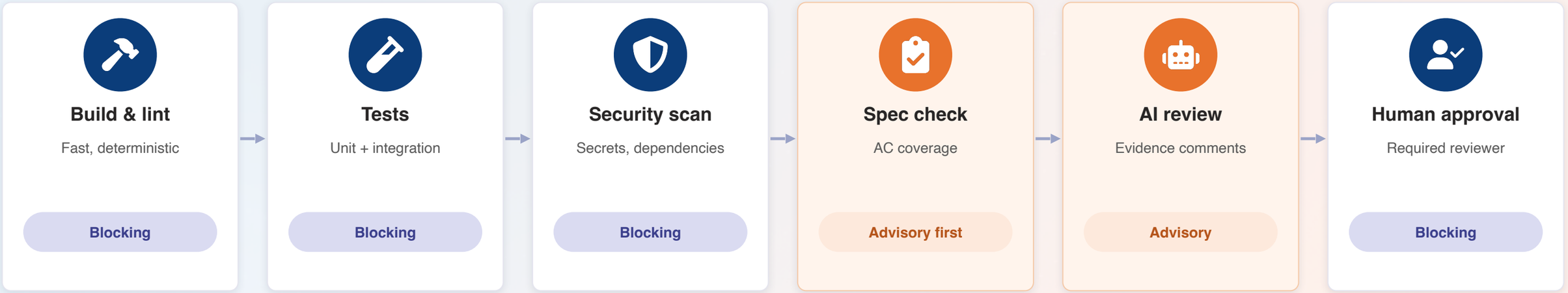
AI use disclosed

Review depth matches origin

Tip: keep the template under a minute to complete. Long templates get ticked without being read.

Quality Gates: Layered Checks Before a Human Looks

Cheap, deterministic gates run first; costly, probabilistic ones run last.



Cost and variability rise to the right → so do false alarms.

Design rule: block only on checks you trust. Start a new AI-based gate as advisory, measure how often it is right, and promote it to blocking only when the team agrees it earns it.

Real-World Gate Recipes

Each gate answers one question and has an owner and a defined response.

GATE	CONDITION	RESPONSE
Changed code has tests	Coverage on changed lines meets a team-set threshold	Block or advise — team decides
No secrets or risky dependencies	Scanner finds none at agreed severity	Block
Every AC maps to a test	Spec check lists AC IDs with no test	Advise → block when trusted
Large change flagged	Diff exceeds the team's size guide	Advise: ask to split
Protected area touched	Auth, payments or config paths changed	Require named reviewer

No universal numbers: coverage, size and severity thresholds are for your team to set from its own history. Revisit them quarterly.

LLM-as-Judge: A Model That Grades Another Model's Work

Use it when “is this good?” needs judgement that a rule or a test cannot express.

Inputs

Task or spec, output under review, and the rubric

Judge model

Applies the rubric criterion by criterion

Score + rationale

Per-criterion score with cited evidence

Action

Pass, advise, or escalate to a human

USED IN CODE REVIEW TO JUDGE…

- Does the PR meet the spec?
- Is the AI reviewer's comment correct and useful?
- Is the PR description accurate?
- Is the generated test meaningful?

Remember: a judge is another model. It can be wrong, biased or inconsistent, so it is measured and calibrated like any other check — which is the next three slides.

Designing a Judge Rubric: Anchored, Specific, Few Criteria

Vague rubrics give vague scores. Describe what each score level looks like.

CRITERION	SCORE 1	SCORE 3	SCORE 5
Spec compliance	Misses a stated criterion	Meets most; gaps unstated	Every AC met, with evidence
Test evidence	No tests for new behaviour	Tests exist; weak assertions	Tests assert each AC and edge
Risk awareness	Ignores callers and rollback	Mentions risk generically	Names callers, impact, rollback
Comment quality	Vague, no location	Located but no fix	Located, evidenced, actionable

Ask for a score and a one-line evidence quote per criterion

Use a small discrete scale (e.g. 1–5), not free text

Include one good and one bad example per criterion

Version it: treat the rubric prompt like code. When scores shift, you can trace it to a rubric change.

Where Judges Go Wrong — and What Reduces It

Four documented failure modes. None disappear; all can be reduced and monitored.

FAILURE MODE	MITIGATION
Length bias — Longer answers score higher	Reward concision in the rubric; compare scores to length
Position bias — The first option in a pair is favoured	Randomise or swap order; score each output alone
Self-preference — A model favours its own family's output	Use a different model family as judge
Confident-but-wrong — Assured wording outscores hedged truth	Require cited evidence; check against human labels

Practical stance: a judge is a cheap, consistent first pass — not ground truth. Spot-check it against human decisions regularly.

Calibrate the Judge, Then Route by Confidence

Trust is earned by measuring agreement with humans — and by knowing when to hand over.

CALIBRATION LOOP

Gold set
Humans label real PRs

→

Run judge
Same PRs, same rubric

→

Compare
Agreement vs humans

→

Adjust
Tighten rubric, add examples

Agreement is often measured with a statistic such as Cohen's kappa. The target is a team decision; re-check when the workload or model changes.

ROUTING BY CONFIDENCE (illustrative policy)

High confidence, low risk
Pass to human with summary

Medium confidence
Human reviews flagged items

Low confidence or security finding
Escalate to named reviewer

Closing the loop: a human approves every merge. Track how often reviewers overturn the judge — that rate tells you whether it is helping or just adding noise.

Lab: Review a Flawed Change

LAB 1 · 15 min

Run AI review and a judge on a deliberately flawed PR, then close the loop as the human.

Goal Configure a PR quality flow, run AI review and a judge against a flawed change, and capture the evidence.

STEPS

1. Add the PR template and one instruction file with 5–8 rules from your spec and standards. Open a PR containing a flawed change (provided).
2. Request Copilot code review. Record each comment: did it give a location, evidence and a fix?
3. Run the judge prompt with the rubric on the same PR. Compare its scores with the AI review and your own view.
4. Decide as the human: approve, request changes, or escalate. Note review time and any defect the tools missed.

FILES YOU CREATE

```
repo/
lab08/
  pull_request_template.md
  .github/instructions/
  judge-rubric.md
  review-evidence.md
```

DONE WHEN

review-evidence.md lists what each reviewer (AI, judge, you) found, what each missed, and your final decision with reasons.

Best Practices for AI Review, Gates and Judges

Automate the checking; keep humans accountable for the deciding.

DO THIS	AVOID THIS
Require a spec link and test evidence in every PR	Treat AI comments as approvals or as ground truth
Write short, specific review rules; test them on real PRs	Pile on vague rules like “be more accurate”
Run cheap deterministic gates before AI-based ones	Block merges on a check nobody has measured
Start new AI gates as advisory; promote on evidence	Let one judge score its own model's output
Use anchored rubrics and a different-family judge	Skip human spot-checks of the judge
Escalate low confidence and security findings to a human	Chase review speed while quality signals get worse

Golden rule: the goal is fewer review cycles at the same or better quality — measure both, never just the first.

What You Can Do Now

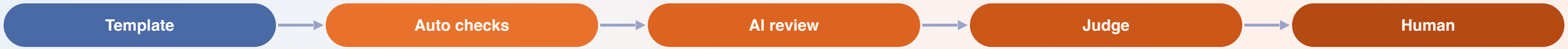
Six outcomes — a PR flow you can trust because you can measure it.

- ✓ Place AI review, gates and a judge in a clear review pipeline
- ✓ Configure Copilot review with instructions and a PR template
- ✓ Ask for evidence-based comments with confidence and escalation
- ✓ Layer quality gates and choose blocking versus advisory
- ✓ Write an anchored LLM-as-Judge rubric and know its biases
- ✓ Calibrate the judge and keep a human approving every merge

NEXT: MODULE 09 — STUDIO-STYLE NO-CODE AGENTS (ROVO AGENT STUDIO) · 2 HOURS

See how product and program roles author governed agents in Atlassian Rovo Studio, and where that meets engineering.

TODAY'S REVIEW PATH



Questions & Discussion

Module 08 — PR Automation, Quality Gates & LLM-as-Judge